1. Phase 1: Complete CRUD Operations

Begin by designing a core Book model entity containing fields for an ID, title, author, isbn, and price. Create an extended `BookRepository` interface using Spring Data JPA, and set up a `BookController` to handle the following API interactions.

Tasks:

1. **Implement Book Creation and Retrieval (`POST /api/books` & `GET /api/books`):** Create the foundational endpoints to save new books to the database and retrieve all existing books.
2. **Implement Book Updates (`PUT /api/books/{id}`):** Add an endpoint that accepts a modified Book object payload in the request body. If the book exists, update its title, author, isbn, and price, then save it. If the target resource ID does not exist, return a `404 Not Found` response.
3. **Implement Book Deletion (`DELETE /api/books/{id}`):** Add an endpoint that targets a book by its database ID and removes it from the record system. Ensure it returns a `204 No Content` status code upon successful removal.
4. **Custom Finder Query:** Leverage Spring Data JPA's automatic method derivation capability. Implement a new controller route `GET /api/books/isbn/{isbn}` that searches the repository layer using a custom method signature `findByIsbn(String isbn)`.

> **Architecture Pro-Tip:**
> Always favor constructor-based dependency injection inside your Spring Controllers over field injection style. This encourages immutability and makes your classes easier to unit-test.

2. Phase 2: Centralized Global Exception Handling

Currently, whenever a validation failure occurs (e.g., negative book prices) or a resource is missing, the Spring application throws standard framework stack traces. You must create a cleaner API client interface.

Tasks:

- Create a dedicated global handler package and declare a class annotated with `@RestControllerAdvice`.
- Write an exception interceptor specifically targeting `MethodArgumentNotValidException`. Extract all failed parameters and compile them into a uniform structure containing field names and messages, accompanied by an explicit `400 Bad Request` HTTP status.
- Create a custom runtime exception named `ResourceNotFoundException`. Wire your global advice handler to catch this exception and respond automatically with a clean `404 Not Found` status and an explicit error explanation string.

3. Phase 3: Relational Data Mapping (Advanced)

In a real bookstore application, books do not exist in isolation. You must demonstrate an understanding of data relational mappings within JPA by linking books to their respective authors.

Tasks:

1. Design and implement a new `Author` entity containing fields for `Long id`, `String firstName`, `String lastName`, and `String biography`.
2. Establish an operational relational mapping link: A book belongs to a single author, while an author can write multiple books. Add a `@ManyToOne` mapping inside your `Book` class linking to the new `Author` entity.
3. Create an `AuthorRepository` and a corresponding `AuthorController` to handle basic CRUD lifecycle events for Authors independently before assigning them to books.

4. Submission Guidelines

Commit your completed solution codebase directly into a private GitHub repository. Share the repository access link or invite your reviewer directly via the GitHub platform. Ensure your final repository features a structured commit history tracking your implementation phases and includes a brief description of how to run the application using Maven.